

Backend Interview Questions

The Full Answer Guide

80 questions across 10 topics — with the concept, a concrete example, an analogy, and the specific line that signals a senior answer instead of a memorized one.

Built for a 3+ year backend engineer prepping for product-company interviews.
Companion to your uploaded question bank — same 10 topics, same 80 questions, full answers.

How to actually use this

Read CONCEPT and ANALOGY first, close the page, and say the answer out loud in your own words. If you can't, you don't know it yet — you recognized it. Recognition fails under interview pressure; recall doesn't.

The EXAMPLE in each answer is deliberately generic. Before your interviews, swap in a real instance from your own systems — the routing engine, the billing migration, anything you've actually touched. An interviewer can tell the difference between a textbook example and a lived one in about one sentence.

SENIOR SIGNAL is not a bonus fact — it's the one sentence that decides whether the interviewer writes "solid" or "strong hire" on the scorecard. Say it, don't just know it.

Prioritize the five-topic core first: Indexing, Transactions, Caching, API Design, and Production Debugging. They show up in almost every backend loop.

Contents

01	Database Indexing	ASKED EVERYWHERE	8 Q
02	Transactions & Isolation Levels	ASKED EVERYWHERE	8 Q
03	Concurrency	VERY COMMON	8 Q
04	Caching	ASKED EVERYWHERE	8 Q
05	Idempotency	VERY COMMON	8 Q
06	Message Queues	VERY COMMON	8 Q
07	Distributed Systems Basics	VERY COMMON	8 Q
08	API Design	ASKED EVERYWHERE	8 Q
09	Database Design	VERY COMMON	8 Q
10	Production Debugging	ASKED EVERYWHERE	8 Q

01 Database Indexing

ASKED EVERYWHERE

The #1 performance topic in backend interviews. The bar isn't the definition — it's whether you've run EXPLAIN on a real slow query and fixed it.

1 What is a B-tree index and how does it work internally?

CONCEPT A B-tree (balanced tree) index stores column values in sorted order across a multi-level tree of pages. Each node holds sorted keys with pointers to child pages or to the actual rows. A lookup compares against a small set of keys per level and descends — that's what makes it $O(\log n)$ instead of $O(n)$.

EXAMPLE An index on users.email in Postgres: searching WHERE email = 'x' walks root to branch to leaf pages using binary search per page — roughly 3-4 page reads for a million rows, instead of scanning all of them.

ANALOGY A phone book split into volumes with tab dividers. You don't read every page — you jump straight to the right volume, then the right letter, then the right sub-range.

SENIOR SIGNAL Say "sorted data structure," not "makes queries faster." The interviewer wants the mechanism, not the effect.

2 What is the difference between a clustered and non-clustered index?

CONCEPT A clustered index physically orders the table's rows by the index key — it IS the table's storage, so there's only one per table. A non-clustered index is a separate structure holding key + pointer back to the actual row.

EXAMPLE In MySQL InnoDB the primary key is always the clustered index. A secondary index on email stores email plus the primary key — so a lookup by email means: find the PK in the secondary index, then do a second lookup in the clustered index. Two hops, not one.

ANALOGY Clustered = a library shelved by ISBN; the shelf order is the index. Non-clustered = a card catalog by author name that points you to the ISBN shelf.

SENIOR SIGNAL Naming the "extra hop" for a secondary-index lookup in InnoDB is the detail that shows real depth, not textbook recall.

3 What is a composite index? Explain the left-prefix rule.

CONCEPT An index on (A, B, C) is sorted first by A, then by B within each A, then C within each B. A query can use it only if it filters on a leading, contiguous prefix: A alone, A+B, or A+B+C. Filtering on B alone can't use it.

EXAMPLE Index (user_id, created_at) speeds up WHERE user_id = 5 AND created_at > X, and WHERE user_id = 5 alone — but not WHERE created_at > X alone.

ANALOGY A phone book sorted by (last name, first name). You can jump straight to "Smith" or "Smith, John" — but you can't jump to all the "Johns" without scanning the whole book.

SENIOR SIGNAL Explaining WHY B alone fails (its values are scattered inside every A group) is what separates understanding from memorizing the rule.

4 When would you NOT add an index?

CONCEPT Every index adds write cost — it must be updated on every INSERT, UPDATE, and DELETE — plus storage. Skip indexing: high-write, low-read tables; low-cardinality columns where the planner may ignore the index anyway; and small tables where a full scan is already cheap.

EXAMPLE A boolean is_deleted column that's 95% false. An index barely narrows the result set, so Postgres's planner often ignores it in favor of a sequential scan.

ANALOGY Adding a table of contents to a 10-page pamphlet — it costs more to maintain than it ever saves you flipping pages.

SENIOR SIGNAL Naming the write-cost trade-off explicitly beats a vague "it depends."

5 What is a covering index?

CONCEPT An index that contains every column a query needs — in SELECT, WHERE, and ORDER BY — so the engine answers entirely from the index without touching the table at all (no extra "bookmark lookup").

EXAMPLE Index (user_id, status, created_at) fully covers SELECT status, created_at FROM orders WHERE user_id = 5 — zero table hits.

ANALOGY A restaurant menu with prices and calories already printed on it, so you never need to ask the kitchen for extra info.

SENIOR SIGNAL Tie it explicitly to read-heavy API latency: "it avoids the extra table I/O." That's the connection interviewers listen for.

6 How do you diagnose a slow query in production?

CONCEPT Run EXPLAIN ANALYZE. Look at actual vs. estimated row counts and the scan type — a Seq Scan on a large table is the red flag. Then verify any fix by re-running with real timing, not just checking that the plan shape changed.

EXAMPLE A dashboard query taking 4s: EXPLAIN shows a Seq Scan on an 8M-row orders table filtering on customer_id. Adding an index on customer_id flips it to an Index Scan; EXPLAIN ANALYZE confirms 40ms.

ANALOGY A doctor orders the test (EXPLAIN) before prescribing (adding an index) — not the other way around.

SENIOR SIGNAL Say you verify with actual query time after the fix, not just "I'd add an index." That shows you don't guess.

7 What is index cardinality and why does it matter for query planning?

CONCEPT Cardinality is the number of distinct values relative to row count. High cardinality (emails, UUIDs) means the index narrows results a lot, so it's useful. Low cardinality (a status with 3 values) means it barely narrows anything, so the planner may skip the index for a seq scan.

EXAMPLE An index on country (5 distinct values, 10M rows) is usually ignored by the planner in favor of a seq scan, unless one value is genuinely rare.

ANALOGY A book index is only useful if each entry points to a few pages. If every entry points to half the book, you might as well read it cover to cover.

SENIOR SIGNAL Tie cardinality to what the query planner's cost estimator actually does with it — not just the definition.

How does an index affect INSERT, UPDATE, and DELETE performance?

CONCEPT Every write must also update every index on that table — B-tree rebalancing, page splits, the works. More indexes means more write amplification and more lock contention.

EXAMPLE A table with 10 indexes on a hot write path (order_items during checkout) can see write latency spike under load even though reads stay fast.

ANALOGY Every extra card catalog you maintain in a library is one more thing to re-file every time a book moves shelves.

SENIOR SIGNAL Frame it explicitly as a read/write trade-off, and mention auditing unused indexes (pg_stat_user_indexes) before adding more.

02 Transactions & Isolation Levels

ASKED EVERYWHERE

One of the most misunderstood backend topics. Most developers use @Transactional daily but can't name their app's actual isolation level or what bugs it permits.

1 What are ACID properties? Explain each one with a real example.

CONCEPT Atomicity: all-or-nothing — a failed step rolls back everything. Consistency: the DB moves between valid states per its own constraints (FKs, checks). Isolation: concurrent transactions don't see each other's uncommitted work. Durability: once committed, it survives a crash (WAL + fsync).

EXAMPLE A funds transfer: debit account A, credit account B. Atomicity guarantees that if the credit fails, the debit is rolled back too — you never end up with money debited but not credited anywhere.

ANALOGY A bank vault transaction — either both drawers open and close together, or neither does.

SENIOR SIGNAL Give the transfer example unprompted. It shows you've internalized the concept, not just memorized the acronym.

2 What are the 4 isolation levels? What problem does each one prevent?

CONCEPT READ UNCOMMITTED (sees uncommitted/dirty data — rarely used). READ COMMITTED (only sees committed data, but a repeated read within the same transaction can still change — Postgres and Oracle's default). REPEATABLE READ (rows stay consistent for the whole transaction — MySQL's default). SERIALIZABLE (transactions behave as if run strictly one after another).

EXAMPLE Postgres defaults to READ COMMITTED — two reads of the same row inside one transaction can return different values if another transaction commits an update in between.

ANALOGY Each level is really about how "frozen in time" your view of the database stays while you work.

SENIOR SIGNAL State your database's actual default from memory (MySQL = REPEATABLE READ, Postgres = READ COMMITTED). That's real experience talking, not a textbook.

3 What is a dirty read, phantom read, and non-repeatable read?

CONCEPT Dirty read: reading another transaction's uncommitted change, which can vanish on rollback. Non-repeatable read: the same row, read twice in one transaction, returns different values because another transaction committed an update in between. Phantom read: a range query returns a different row SET on re-run because rows were inserted or deleted.

EXAMPLE Phantom read: `SELECT COUNT(*) WHERE status = 'pending'` returns 10, then 12 moments later, because new orders were inserted mid-transaction.

ANALOGY Dirty read is reading someone's rough draft before they hit save. Phantom read is counting empty seats in a theater while people are still walking in.

SENIOR SIGNAL Distinguish non-repeatable (same row, different value) from phantom (different row set entirely) — it's the pair candidates confuse most.

4 What is a deadlock? How do you detect and prevent one?

CONCEPT Two transactions each hold a lock the other one needs, so both wait forever. The database's deadlock detector picks a victim and aborts it (rolled back with an error) so the other can proceed.

EXAMPLE Transaction A locks row 1, then wants row 2. Transaction B locks row 2, then wants row 1. Classic deadlock — common in fund transfers if you don't lock accounts in a consistent order.

ANALOGY Two cars meeting nose-to-nose in a single-lane alley, each waiting for the other to back up first.

SENIOR SIGNAL Mention consistent lock ordering (always lock the lower ID first) as the real prevention — not just "keep transactions short."

5 What is the difference between optimistic and pessimistic locking?

CONCEPT Pessimistic locking locks the row upfront (SELECT ... FOR UPDATE), blocking everyone else until you're done — safe under high contention, but costs throughput. Optimistic locking doesn't lock at all; it reads a version number, and on write checks whether the version still matches, retrying if someone else changed it first.

EXAMPLE An e-commerce inventory decrement under low contention uses a version column (JPA's @Version) — it fails fast on conflict instead of blocking every reader.

ANALOGY Pessimistic is reserving a meeting room before you start talking. Optimistic is just starting the meeting and checking afterward if someone double-booked it.

SENIOR SIGNAL Tie the choice to contention level, not "optimistic is just better." Under high contention, optimistic locking causes retry storms.

6 When would you use SERIALIZABLE isolation and what does it cost you?

CONCEPT SERIALIZABLE guarantees transactions behave as if executed strictly one at a time, eliminating every anomaly including phantoms. The cost: the DB detects conflicts and aborts one side (a serialization failure), forcing app-level retries — throughput drops sharply under contention.

EXAMPLE A ledger computing account balances via aggregate queries might need SERIALIZABLE to avoid phantom rows skewing the total — but it's rarely the default because most apps can't tolerate the retry overhead at real scale.

ANALOGY Making every customer in a shop check out one at a time through a single register — completely safe, painfully slow.

SENIOR SIGNAL State just as clearly when you WOULDN'T use it as when you would. That's judgment, not "safer is always better."

7 What is a two-phase commit and what problem does it solve?

CONCEPT Coordinates atomicity across multiple databases or services. Phase 1 (prepare): the coordinator asks every participant "can you commit?" — they lock resources and reply yes or no. Phase 2 (commit): if everyone said yes, the coordinator tells everyone to commit; if anyone said no, everyone rolls back.

EXAMPLE A payment spanning two database shards (or a DB plus a message broker) uses 2PC to keep both in sync — but if the coordinator crashes mid-phase-2, participants can be left blocked, still holding locks.

ANALOGY A wedding officiant asking both people "do you?" privately before either says "I do" out loud — no one commits until everyone has silently agreed.

SENIOR SIGNAL Mention the blocking failure mode on coordinator crash — it's exactly why most systems now prefer sagas or eventual consistency over 2PC.

8

What is a savepoint? When would you use one?

CONCEPT A named point inside a transaction you can roll back to without aborting the whole transaction — statements after it are undone, but earlier statements and the outer transaction stay open.

EXAMPLE A batch import processes 100 rows in one transaction. Before each row, set a savepoint — if row 47 fails validation, roll back to that savepoint and continue with row 48 instead of losing the whole batch.

ANALOGY A checkpoint in a video game level — dying resets you to the checkpoint, not to the start of the game.

SENIOR SIGNAL Give the batch-processing example. It's the real use case interviewers expect, not an abstract one.

03 Concurrency

VERY COMMON

The hardest bugs to reproduce and the most expensive in production. This topic tests whether you understand what actually happens when two requests hit the same code in the same millisecond.

1 What is a race condition? Give a real backend example.

CONCEPT The outcome depends on the timing of unsynchronized operations touching shared state — two threads or requests read-modify-write the same value without coordination, and one update is silently lost.

EXAMPLE Two requests read account balance = 100. Both compute $100 - 50 = 50$. Both write 50 back. The real balance should be 0, but one deduction vanished — a classic lost update. Shows up in payments, inventory, and counters.

ANALOGY Two people editing the same spreadsheet cell offline and both saving — the last save wins, and the first edit disappears without a trace.

SENIOR SIGNAL Use the balance example unprompted. "Lost update" is the exact term interviewers are listening for.

2 What is the difference between concurrency and parallelism?

CONCEPT Concurrency is structuring a program to handle multiple tasks — interleaved, possibly on a single core, making progress on many things without necessarily doing them at the same instant. Parallelism is literally executing multiple tasks at the same instant, on multiple cores.

EXAMPLE Node.js handles 10,000 open connections concurrently on one thread via the event loop. A Java thread pool with 8 threads on an 8-core machine achieves true parallelism for CPU-bound work.

ANALOGY Concurrency is one chef juggling five pans, switching between them. Parallelism is five chefs at five pans, all at once.

SENIOR SIGNAL Note you can have concurrency without parallelism (single-core async) — that's exactly the distinction most people blur.

3 How does Node.js handle concurrent requests without multiple threads?

CONCEPT A single-threaded event loop plus non-blocking I/O. The main thread never blocks on I/O — a DB call, file read, or network request is handed to the OS or libuv's thread pool with a registered callback, and the loop keeps processing other requests until that I/O completes and the callback is queued.

EXAMPLE 1,000 concurrent DB-bound requests are handled fine, since each `await db.query()` frees the loop immediately. 1,000 CPU-bound requests (like image resizing) queue up and block everyone, since there's only one thread doing actual JS execution.

ANALOGY One waiter taking orders from every table and handing tickets to the kitchen, instead of standing at the kitchen window waiting for each dish.

SENIOR SIGNAL Flag the CPU-bound exception. Knowing when Node's model breaks down matters more than reciting the happy path.

4 What is an atomic operation? Why does it matter for counters?

CONCEPT An operation that executes as a single indivisible step — nothing can interleave mid-way. It matters for counters because "read, add 1, write" is three separate steps unless the DB or cache guarantees them as one.

EXAMPLE Redis INCR is atomic (single command, single-threaded execution). SET counter (GET counter)+1 written in application code is NOT atomic — two clients can read the same value and stomp on each other's increment.

ANALOGY Atomic is flipping a single light switch. Non-atomic is "check if the light is off, then walk over and flip it" — someone else can flip it in between.

SENIOR SIGNAL Name the exact anti-pattern — read-then-write in application code — that's precisely the gap this question is probing for.

5 What is a connection pool and what happens if you don't use one?

CONCEPT A pre-opened, reused set of DB connections shared across requests, instead of a fresh TCP-plus-auth handshake per request. Without one, every request pays that overhead, and under load the DB hits max_connections and starts rejecting new ones.

EXAMPLE HikariCP in a Spring Boot app maintains roughly 10-20 live connections; 500 concurrent requests share and queue for them instead of trying to open 500 raw DB connections.

ANALOGY A pool of taxis waiting at a rank, versus every passenger calling a dealership to buy a new car for a single ride.

SENIOR SIGNAL Mention pool sizing — too large can overwhelm the DB, too small causes request queuing. That shows you've tuned one, not just used the default.

6 What is a distributed lock and when do you need one?

CONCEPT A lock that's visible and enforced across multiple service instances or processes, not just within one JVM. You need it when several instances might race to process the same job or mutate the same shared resource.

EXAMPLE Two instances of a cron-triggered "send daily report" job both fire at 9:00 AM — a Redis SETNX lock with a TTL ensures only one instance actually runs it.

ANALOGY One key to a single supply closet, shared by workers across multiple store locations.

SENIOR SIGNAL Mention the TTL and its failure mode — a lock holder dying without releasing, leaving a stale lock — showing you know the naive version is broken.

7 What is the thundering herd problem in the context of concurrency?

CONCEPT A large number of waiters — threads, cache clients, cron triggers — all become unblocked or wake up at the exact same moment and hit a shared resource simultaneously, overwhelming it.

EXAMPLE A popular cache key expires and 5,000 concurrent requests all miss the cache at once, slamming the DB with the same expensive query simultaneously.

ANALOGY One alarm going off and an entire dormitory floor sprinting for the single bathroom at the same time.

SENIOR SIGNAL Name an actual fix — a single-flight lock on rebuild, or jittered TTLs — rather than just describing the problem.

How do you make a counter increment safe under high concurrency?

CONCEPT Never do read-modify-write in application code under concurrency. Use a DB atomic update (`UPDATE t SET count = count + 1`), a cache atomic command (Redis `INCR`), or route all increments through one serialized service or queue.

EXAMPLE A "view count" feature uses `UPDATE posts SET views = views + 1 WHERE id = ?` instead of fetching the count, adding 1 in Java, then saving it back.

ANALOGY Telling the bank "add \$10 to my account" instead of asking your balance, doing the math yourself, and telling them the new total.

SENIOR SIGNAL State the DB-level vs. app-level increment distinction directly — it's exactly what's being tested.

04 Caching

ASKED EVERYWHERE

In almost every senior backend interview. The wrong answers are "use Redis" and "add a TTL." The right answers involve strategy, invalidation, and failure modes.

1 Explain the four main caching strategies and when to use each.

CONCEPT Cache-aside: the app checks the cache, and on a miss reads the DB and populates the cache — simple and most common, but the first request after expiry is slow. Write-through: every write goes to the cache and DB together — always fresh, but adds write latency. Write-behind: write to the cache immediately, flush to the DB asynchronously — fast writes, but risks data loss if the cache dies before flushing. Refresh-ahead: proactively refresh hot keys before they expire — avoids stampedes, but wastes cycles on keys that go cold.

EXAMPLE A product catalog page uses cache-aside. A session store might use write-through, since staleness there is unacceptable.

ANALOGY Cache-aside is checking the fridge, cooking if it's empty, then restocking. Write-through is restocking the fridge the moment you buy groceries.

SENIOR SIGNAL Name the failure mode of each strategy, not just its mechanism — that's what separates this from a dictionary-definition answer.

2 What is cache invalidation and why is it considered one of the hardest problems in CS?

CONCEPT Deciding exactly when a cached value becomes wrong and needs to be removed or updated. Too aggressive (short TTLs, eager invalidation) means more misses and more DB load. Too lax means stale data reaches users. There's no universal rule — it depends entirely on how much staleness the feature can tolerate.

EXAMPLE A stock ticker price can't tolerate more than a second of staleness; a user's "about me" bio can tolerate minutes without anyone noticing.

ANALOGY Deciding how often to reprint a restaurant menu — too often is wasteful, too rarely means quoting old prices.

SENIOR SIGNAL Say "it depends on the staleness tolerance of the feature" explicitly — that's the senior framing versus a junior "just use a TTL."

3 What is a cache stampede / thundering herd and how do you prevent it?

CONCEPT When a hot key expires, many concurrent requests miss at once and all hit the DB to rebuild it simultaneously. Prevention: a mutex/lock so only one request rebuilds while others wait or get stale data; probabilistic early expiry (refresh slightly before real expiry, randomized); or staggered TTLs so keys don't all expire in sync.

EXAMPLE A homepage "trending" cache key used by every visitor expires at once during a traffic spike, and the DB gets hammered by thousands of identical rebuild queries in the same second.

ANALOGY A bakery's opening-time rush, except every customer's alarm was set to the exact same minute.

SENIOR SIGNAL "Lock on rebuild" is the specific phrase interviewers want to hear — say it explicitly, not just "handle it somehow."

4 What is a hot key problem in Redis and how do you solve it?

CONCEPT One key receives disproportionate traffic — a celebrity's profile, a flash-sale item — and overloads the single Redis node or shard that owns it, even though the cluster overall has plenty of capacity.

EXAMPLE A flash-sale product's stock-count key gets 50k reads per second, saturating one shard while every other shard sits idle.

ANALOGY One checkout lane at a store getting a stampede while nine others stand empty.

SENIOR SIGNAL Mention local in-process caching of hot keys as the first line of defense — it's cheaper than resharding the whole cluster.

5 When should you NOT cache something?

CONCEPT Skip caching data that changes faster than any reasonable TTL, values that must always be exact (account balances, inventory at checkout), per-user data that rarely repeats (low hit rate isn't worth the complexity), or anything where stale data creates a security or compliance issue.

EXAMPLE Don't cache a "remaining seats" count at the final step of checkout — always read that fresh from the DB.

ANALOGY You don't write "current weather" on a printed sign — by the time someone reads it, it's already wrong.

SENIOR SIGNAL Give the checkout/inventory example. It's the canonical case where caching would be a bug, not an optimization.

6 What is the difference between a local cache and a distributed cache?

CONCEPT Local (in-process, e.g. Caffeine) is fastest — no network hop — but each instance holds its own copy, so it's inconsistent across a fleet. Distributed (Redis) gives a shared, consistent view across instances, at the cost of a network round trip.

EXAMPLE Cache a parsed config or feature-flag set locally per instance, since it rarely changes and low staleness there is cheap. Cache a shopping cart in Redis so it's visible from any instance behind the load balancer.

ANALOGY Local is notes in your own notebook. Distributed is a shared whiteboard everyone in the room can see and edit.

SENIOR SIGNAL Give one real use case for each — that shows you've actually chosen between them, not just defined the terms.

7 How do you handle cache consistency when data changes?

CONCEPT Event-driven invalidation: publish an event on write, and every consumer evicts or updates its copy — scales well, has propagation delay. Write-through: update the cache synchronously with the DB write — simplest, adds write latency. Versioned keys: a write creates a new key, and the old key simply expires — avoids races entirely, at the cost of temporary duplication.

EXAMPLE A catalog updates via a product-price-changed event, and every service instance subscribed to that topic evicts its local cache entry.

ANALOGY Versioned keys are like publishing "menu v2" instead of erasing and rewriting the old chalkboard mid-service.

SENIOR SIGNAL Mentioning versioned keys — the least obvious of the three — signals more than surface-level Redis experience.

8

What is a CDN and how is it different from an application-level cache?

CONCEPT A CDN caches static or cacheable HTTP responses at edge locations physically close to users, reducing network latency and origin load for content like images, JS bundles, or cacheable API responses. An application cache (Redis) sits next to your backend and caches computed data or DB query results, reducing load on your database rather than latency to the user.

EXAMPLE A CDN caches `/product-images/*` and a fully rendered public product page; Redis caches the result of "get user cart by ID."

ANALOGY CDN is a vending machine on every floor of the building. App cache is a well-stocked storeroom behind the one shop.

SENIOR SIGNAL Say plainly: "CDN reduces latency, app cache reduces DB load." That one-liner is exactly what's being tested.

05 Idempotency

VERY COMMON

A senior-level topic most juniors have never heard of and most mid-levels know the word for but can't implement. Comes up in every payments, messaging, and distributed systems interview.

1 What is idempotency? Why does it matter for APIs?

CONCEPT Calling an operation once or N times produces the same end state. It matters because networks are unreliable — clients time out and retry — and without idempotency, a retry can duplicate a side effect: charge twice, send two emails, create two orders.

EXAMPLE A payment API call times out on the client side, even though the server actually succeeded. The client retries "pay \$50" — without protection, the customer is charged \$100.

ANALOGY Pressing an elevator call button five times because you're not sure it registered — it should still send one elevator, not five.

SENIOR SIGNAL Anchor the "why" specifically in retries, not just an abstract "so calling it twice is safe."

2 Which HTTP methods are idempotent by spec? Which aren't?

CONCEPT GET, PUT, DELETE, and HEAD are idempotent by spec — repeating them leaves the resource in the same state as calling once. POST is not — each call typically creates a new resource. PATCH isn't idempotent by spec by default, though it can be designed to be.

EXAMPLE PUT /users/5 {name: "Avnish"} sets the name regardless of how many times you call it. POST /users creates a new user on every single call.

ANALOGY PUT is like setting a thermostat to 70°F — do it once or ten times, the room still ends up at 70. POST is like adding a log entry — ten calls, ten entries.

SENIOR SIGNAL Know that PATCH is the gray area. Most candidates only know GET/POST/PUT/DELETE and miss this nuance entirely.

3 How do you implement idempotency for a payment API endpoint?

CONCEPT The client generates a unique idempotency key (a UUID) per logical operation and sends it in a header. The server checks whether that key was seen before: if yes, return the stored result without re-executing; if no, process the request and store the key plus result together, atomically, before returning.

EXAMPLE Stripe's API requires an Idempotency-Key header on POST /charges — retrying with the same key returns the original charge's result, not a second charge.

ANALOGY A claim ticket at a coat check — hand in the same ticket twice, you get the same coat back, not two coats.

SENIOR SIGNAL Mention storing the key BEFORE processing completes, not after. That ordering is the actual crux of the design.

4 How do you store and look up idempotency keys efficiently?

CONCEPT Redis with a TTL is the standard choice — O(1) lookup, and automatic expiry so old keys don't accumulate forever. Store key → {status: processing/done, response}. Use a lock (SETNX) during processing so a second concurrent request with the same key waits instead of racing.

EXAMPLE Key idem:txn-abc123 maps to {"status":"done","response":{...}} with a 24h TTL. A concurrent duplicate request sees status: processing and blocks or polls instead of re-executing.

ANALOGY A "table reserved" sign you put up the moment you start seating someone, not after they've finished eating.

SENIOR SIGNAL Locking during processing — not just checking if the key exists — is what distinguishes a real implementation from a naive one.

5 What happens if the server processes the request but crashes before responding?

CONCEPT If the server wrote the idempotency key and result before, or atomically with, the actual side effect, a retry with the same key finds the stored result and returns it safely — no reprocessing. If it only wrote the key after fully finishing, a crash mid-way means the retry reprocesses — which is exactly why key-write timing matters.

EXAMPLE Payment charged, then the DB write of "key → charged" fails right after due to a crash. The client retries, the server sees no key, and might double-charge. The fix ties the key write and the charge into one atomic transaction, or leans on the payment processor's own idempotency key.

ANALOGY Signing the delivery receipt at the same moment you hand over the package, not after you've already driven away.

SENIOR SIGNAL This question is really testing the ordering detail from the previous two questions from a different angle — answer it consistently.

6 How do you handle idempotency in a message queue consumer?

CONCEPT Message brokers commonly guarantee at-least-once delivery, so a consumer can receive the same message more than once. The consumer stores processed message IDs (DB or Redis) and checks before processing: if seen, ack and skip; if new, process then record the ID.

EXAMPLE A Kafka consumer processing order-placed events stores the message key or offset in a processed_events table with a unique constraint, so a redelivered message is a no-op insert — a duplicate-key error caught and skipped.

ANALOGY A guest list at the door — even if the same invitation gets handed over twice, the person is only let in once.

SENIOR SIGNAL Tie it explicitly to "at-least-once delivery" — that shows you understand why duplicates happen, not just that they can.

7 What is the difference between idempotency and exactly-once delivery?

CONCEPT Idempotency is a property of the operation itself — safe to repeat, implemented at the app or data layer. Exactly-once delivery is a guarantee from the messaging system that a message is processed exactly once, no more and no less — genuinely hard and expensive to guarantee end to end. At-least-once delivery plus idempotent consumers is the practical, cheap approximation of exactly-once.

EXAMPLE Kafka's "exactly-once semantics" (transactional producer/consumer) exist but add real overhead. Most teams instead do at-least-once plus idempotent writes and get the same practical outcome.

ANALOGY Exactly-once delivery is trying to guarantee the mail truck drops your letter exactly once. Idempotency is making sure it doesn't matter if it drops two copies.

SENIOR SIGNAL Say idempotency is "cheaper and more practical" — that shows you'd pick the pragmatic option over the theoretically pure one.

8

Can a non-idempotent operation be made safe to retry? How?

CONCEPT Wrap it in check-then-act with a stored idempotency key: before executing, check if this operation, by key, was already done; if not, execute and persist the result under that key; if yes, return the stored result instead of re-running.

EXAMPLE Turning "send welcome email" (not idempotent — resending isn't a no-op) safe by tracking an `email_sent_for_user_id` row before actually calling the mail provider.

ANALOGY Checking off a to-do list item before doing it a second time by accident.

SENIOR SIGNAL This is the same pattern as the payment-endpoint answer, generalized. Showing you can apply it to a different domain is the actual signal.

06 Message Queues

VERY COMMON

Tests whether you understand async architecture, not just whether you've used a tool. Interviewers want to know when you'd reach for a queue and what breaks without one.

1 What is the difference between a message queue and pub/sub?

CONCEPT A queue: one message is consumed by exactly one consumer, and it's removed once processed — work is distributed, not duplicated. Pub/sub: every subscriber gets its own copy of each message — a broadcast, not a distribution. Kafka blends both: topics behave like pub/sub across consumer groups, but within a group it behaves like a queue, since each partition is owned by one consumer.

EXAMPLE A RabbitMQ work queue: 5 workers pull from one queue, each job done by exactly one worker. An SNS/pub-sub topic: one event fans out to 5 independent subscribers, each processing it fully.

ANALOGY Queue is a single ticket line, one person per ticket. Pub/sub is a radio broadcast — every listener hears the same thing.

SENIOR SIGNAL Mention Kafka's hybrid behavior — that detail shows real usage, not textbook knowledge.

2 Explain at-most-once, at-least-once, and exactly-once delivery.

CONCEPT At-most-once: send and don't retry (fire-and-forget) — fast, can silently lose messages. At-least-once: retry until acknowledged — never loses messages, but can deliver duplicates. Exactly-once: each message processed exactly once — the most expensive to guarantee, requiring real coordination and deduplication. Most production systems default to at-least-once plus idempotent consumers.

EXAMPLE A Kafka consumer commits its offset only after successful processing — if it crashes between processing and committing, the message redelivers, which is at-least-once in practice.

ANALOGY At-most-once is shouting an announcement once and moving on. At-least-once is calling someone repeatedly until they pick up — they might catch two calls back-to-back.

SENIOR SIGNAL Land on "at-least-once plus idempotent consumer is the practical default" as your conclusion — that's the industry-standard answer, not just three definitions.

3 What is a dead letter queue and when does a message end up there?

CONCEPT A separate queue where messages get routed after exhausting their retry limit, so one broken message — a "poison pill" — doesn't repeatedly block or crash the consumer or hold up the rest of the queue.

EXAMPLE A malformed JSON event that throws on every parse attempt gets retried 3 times, then routed to orders-dlq, where an engineer inspects it manually instead of it looping forever.

ANALOGY A "returns" bin at a store for items that can't be shelved normally — they're set aside instead of blocking the checkout line.

SENIOR SIGNAL Mention that DLQs need monitoring and alerting — a DLQ nobody watches is just where bugs go to hide.

4 What is backpressure and how do message queues help?

CONCEPT When a consumer can't process messages as fast as they're produced, the gap has to build up somewhere. A queue absorbs that gap by buffering, so the producer isn't blocked and the consumer isn't overwhelmed synchronously — it drains at its own sustainable pace.

EXAMPLE A spike of 10,000 signups triggers 10,000 "send welcome email" jobs. Without a queue, the email service gets hit directly and falls over; with a queue, it drains steadily over a few minutes.

ANALOGY A funnel — water can pour in fast, but it only drips out the bottom at a controlled rate.

SENIOR SIGNAL State explicitly that a queue doesn't remove the work, it smooths the rate — a subtlety many candidates miss.

5 How do you handle duplicate messages in a consumer?

CONCEPT Give every message a unique ID (or use its natural key). The consumer checks a "processed IDs" store before acting — if present, ack and skip; if not, process, then persist the ID, ideally in the same transaction as the side effect to avoid a race.

EXAMPLE A `processed_message_ids` table with a unique constraint on `message_id` — inserting a duplicate throws a constraint violation, which the consumer catches and treats as "already handled."

ANALOGY Checking a guest's name off a list before letting them in, even if their invite got mailed twice.

SENIOR SIGNAL Connecting this explicitly to the idempotent-consumer pattern from the idempotency topic is a strong signal of real understanding.

6 What is a consumer group in Kafka and how does it enable scaling?

CONCEPT A consumer group is a set of consumers that split the work of consuming a topic — each partition is assigned to exactly one consumer within the group at a time, so partitions are the unit of parallelism. Multiple groups can each independently consume the whole topic, which is the pub/sub side.

EXAMPLE A topic with 6 partitions and a group of 6 consumers gets full parallelism. Adding a 7th consumer to that group leaves it idle, since there's no partition left to assign it.

ANALOGY 6 checkout lanes and up to 6 cashiers — extra cashiers past that just stand around with nothing to do.

SENIOR SIGNAL State "max parallelism equals the number of partitions" — that's the number interviewers check you actually know, since it drives real capacity planning.

7 When would you use a message queue instead of a direct API call?

CONCEPT Use a queue when the operation is slow or non-urgent (report generation, email), when you want automatic retry semantics, when producer and consumer need to scale independently, or when you need guaranteed eventual delivery even if the consumer is temporarily down.

EXAMPLE "Generate PDF invoice" gets queued so checkout doesn't wait 5 seconds for rendering. "Check if item is in stock" stays a direct synchronous call, because the user needs the answer immediately.

ANALOGY Mailing a letter (queue — fire it off, it'll get there) versus a phone call (direct call — you need the answer now).

SENIOR SIGNAL Give one example of when you WOULD and when you WOULDN'T use a queue — that shows judgment, not a one-sided answer.

8

What is the difference between Kafka and RabbitMQ?

CONCEPT Kafka is log-based — messages are appended to a persistent, ordered log and retained by a time or size policy regardless of consumption; consumers track their own offset and can replay. High throughput, ordering guaranteed per partition. RabbitMQ is a traditional broker — messages are pushed to consumers and deleted once acknowledged, with more flexible routing (exchanges, topic and fanout patterns) but a lower throughput ceiling.

EXAMPLE Event sourcing or analytics pipelines that need replay and strict ordering reach for Kafka. A task queue for background jobs with complex routing rules often reaches for RabbitMQ.

ANALOGY Kafka is a recorded broadcast archive you can rewind. RabbitMQ is a live phone switchboard routing calls and then hanging up.

SENIOR SIGNAL "Replay" is the single word that best distinguishes them — say it and you've made the core point.

07 Distributed Systems Basics

VERY COMMON

Separates developers who've built on one server from engineers who've thought about what happens when you have many. Nearly every product-company interview includes at least two of these.

1 What is the CAP theorem? Give a real example of a CP and an AP system.

CONCEPT During a network partition, a distributed system must choose between Consistency (every node sees the same, most recent data) and Availability (every request gets a response, even if not the latest data). You can't have all of Consistency, Availability, and Partition tolerance at once when a partition actually happens — and real networks do partition, so it's really a C-vs-A choice under partition.

EXAMPLE Zookeeper and HBase (CP) will refuse to serve a read or write rather than risk returning stale or conflicting data during a partition. DynamoDB and Cassandra (AP) keep serving requests during a partition, accepting that some nodes may return stale data.

ANALOGY CP is a bank refusing a withdrawal if it can't confirm your real-time balance with the main office. AP is an ATM that lets you withdraw based on its last-known balance, reconciling later.

SENIOR SIGNAL Say "partition tolerance isn't really optional" — real networks partition, so the actual choice is C vs. A, not freely picking 2 of 3.

2 What is eventual consistency? Give a real-world example of when it's acceptable.

CONCEPT All replicas will converge to the same value eventually, once updates stop and enough time passes — but at any given instant, different nodes might return different, stale values. Acceptable wherever a slightly stale read doesn't cause real harm.

EXAMPLE Acceptable: a "likes" count on a post lagging by a second or two. Not acceptable: a bank balance check right before approving a withdrawal, or an inventory count at the exact moment of purchase.

ANALOGY Word-of-mouth news spreading through a building — everyone eventually hears it, but not at the same second.

SENIOR SIGNAL Give one acceptable and one unacceptable example unprompted — that shows judgment about where the trade-off is actually safe.

3 What is consistent hashing and why is it used?

CONCEPT Maps both keys and nodes onto a hash ring; a key belongs to the next node clockwise on the ring. When a node is added or removed, only the keys between it and its neighbor need to move — roughly $1/N$ of keys — instead of a full remap as with a simple $\text{hash}(\text{key}) \% N$.

EXAMPLE A Redis cluster or CDN edge-node pool adding a new node during a traffic spike doesn't need to reshuffle the entire cache — most keys stay put.

ANALOGY Seats around a round table — adding one more chair only displaces the people right next to the new chair, not everyone at the table.

SENIOR SIGNAL Contrast it explicitly against $\text{hash} \% N$, which remaps almost everything on resize — that comparison is the whole point of the question.

4 What is a split-brain problem?

CONCEPT A network partition splits a cluster into two or more groups, and each group — unable to see the other — believes it's the sole leader or authority and keeps accepting writes independently. When the partition heals, you're left with two conflicting histories to reconcile.

EXAMPLE A poorly configured leader-election setup where both sides independently promote themselves to primary during a network blip, and both accept writes.

ANALOGY Two branch managers, cut off from head office and each other, both independently declaring themselves CEO and signing contracts.

SENIOR SIGNAL Name the fix — quorum, requiring a majority of nodes to agree before any side accepts writes — not just describe the problem.

5 What is a distributed lock and what are its failure modes?

CONCEPT Same idea as a regular lock, but across processes or services (Redis SETNX, Redlock, or a DB row lock). Failure modes: the lock holder crashes without releasing it (fixed with a TTL so it auto-expires), and clock drift between nodes, which can make a lock appear to expire early on one node while another still thinks it's valid (Redlock addresses this by requiring majority agreement across independent Redis instances).

EXAMPLE A cron job holding a Redis lock crashes mid-run — without a TTL, that lock never releases, and the job never runs again until someone manually intervenes.

ANALOGY A key with an auto-return slot vs. a key that can be lost forever if the borrower disappears.

SENIOR SIGNAL Name both the TTL and the clock-drift failure mode — not just "add a TTL." That's the specific pair interviewers listen for.

6 What is the two-generals problem and why does it matter?

CONCEPT A thought experiment proving two parties can never be 100% certain they've reached agreement over an unreliable channel, since any acknowledgment could itself be lost, requiring an ack of the ack, forever. It's the formal proof that guaranteed exactly-once delivery over an unreliable network is impossible — you can only get arbitrarily close via retries plus idempotency.

EXAMPLE It's the theoretical reason systems settle for at-least-once delivery plus idempotent consumers instead of chasing true exactly-once.

ANALOGY Two generals on separate hills trying to coordinate a simultaneous attack by messenger, but any messenger might be captured — neither can ever be fully sure the other got the message.

SENIOR SIGNAL Connect it back to why at-least-once plus idempotency is the practical industry answer — that link elevates this past trivia.

7 What is a heartbeat in distributed systems?

CONCEPT A periodic "I'm alive" signal a node sends to a coordinator or its peers. If a heartbeat is missed for a configured timeout, the node is presumed dead and the system reroutes traffic, triggers failover, or re-elects a leader.

EXAMPLE Kubernetes liveness probes, or a Kafka broker sending heartbeats to its controller — missing them triggers the broker being marked down and its partitions reassigned.

ANALOGY A lifeguard doing a headcount whistle every few minutes — no response from a swimmer means someone goes to check.

SENIOR SIGNAL Mention the timeout trade-off — too short causes false positives during a blip, too long means slow failure detection. That shows you'd actually tune it.

8

What is the difference between a service mesh and a load balancer?

CONCEPT A load balancer distributes incoming traffic across instances of a service (client to service), typically at L4/L7, sitting at the edge or in front of a service. A service mesh manages service-to-service traffic inside the system — retries, timeouts, circuit breaking, mTLS, tracing — usually via a sidecar proxy next to each instance. They handle different traffic directions and often coexist.

EXAMPLE An ALB or NGINX load balancer routes external user traffic to your API gateway; Istio or Linkerd then manages calls between your internal microservices behind it.

ANALOGY The load balancer is the host seating guests at the restaurant entrance. The service mesh is how the kitchen staff coordinate and hand off dishes internally.

SENIOR SIGNAL "Complements, doesn't replace" is the exact framing to use — many candidates wrongly treat them as competing options.

08 API Design

ASKED EVERYWHERE

Asked in nearly every backend interview because it reveals how you think about contracts, versioning, and the developers who'll consume what you build. A good API is harder to design than to build.

1

What makes an API RESTful? What are the constraints?

CONCEPT Stateless (the server holds no client session state between requests — every request is self-contained), client-server separation, cacheable responses, uniform interface (resource-based URLs plus standard HTTP methods with consistent meaning), a layered system (the client can't tell if it's talking directly to the server or through intermediaries), and optional code-on-demand. Statelessness is the constraint that matters most in practice.

EXAMPLE A REST API shouldn't store "logged-in user" in server memory tied to a session ID — auth state travels with each request, e.g. a JWT in the header, so any server instance can handle any request.

ANALOGY A hotel where every request to the front desk must include your room number and ID — the clerk keeps no memory of who you are between visits.

SENIOR SIGNAL Say why statelessness matters practically: it's what lets you horizontally scale servers without sticky sessions.

2

When would you use REST vs GraphQL vs gRPC?

CONCEPT REST is simple, cacheable, and great for public or standard web APIs, but can over-fetch or under-fetch, needing multiple round trips for nested data. GraphQL lets the client specify exactly the fields it needs in one request — good for mobile clients on constrained bandwidth or complex entity graphs, at the cost of harder caching and more complex server-side resolution. gRPC is a binary protocol (protobuf), very fast, and supports bi-directional streaming — best for internal service-to-service calls where both ends control the schema.

EXAMPLE A public partner-facing API uses REST. A mobile app's feed needing user, post, and comments in one call uses GraphQL. Internal order-service to inventory-service calls use gRPC.

ANALOGY REST is ordering off a fixed menu. GraphQL is a buffet where you pick exactly what you want. gRPC is a private kitchen line only your own staff use, optimized for speed.

SENIOR SIGNAL Give one concrete "I'd pick X because Y" per option rather than just listing pros and cons — that shows decision-making, not memorization.

3

How do you version an API without breaking existing clients?

CONCEPT URL versioning (`/v1/resource`, `/v2/resource`) is the most explicit and easiest for clients to pin to. Header versioning (`Accept: application/vnd.api+json;version=2`) is more "RESTfully pure" but less discoverable. The real rule regardless of scheme: never remove or repurpose a field within a version — only add new optional fields; breaking changes get a whole new version.

EXAMPLE Stripe uses a date-based version header; most product APIs use `/v1/`, `/v2/` in the URL because it's simpler to route and document.

ANALOGY A cookbook edition number on the cover (URL versioning, obvious) vs. a small print note inside changing the recipe (header versioning, easy to miss).

SENIOR SIGNAL "Never remove or rename within a version" is the real substance — say that even if you're only asked about the scheme.

4 What is cursor-based pagination and why is it better than offset?

CONCEPT Offset pagination (LIMIT/OFFSET) gets slower as the offset grows — the DB still scans and skips all prior rows — and is unstable if rows are inserted or deleted mid-paging, causing shifted or duplicated items. Cursor pagination encodes the last-seen row's sort key and fetches "rows after this key," staying fast and stable at any page depth by doing an index seek instead of a scan-and-skip.

EXAMPLE Twitter and Instagram-style infinite feeds always use cursor pagination because offset pagination breaks under a fast-changing feed, where new posts shift every offset.

ANALOGY Offset is "give me rows 10,000 to 10,020" — someone has to count past the first 10,000 every time. Cursor is "give me everything after post #48213" — jump straight there.

SENIOR SIGNAL Mention the stability issue — data shifting mid-page — not just the performance angle. Most candidates only remember the latter.

5 What HTTP status codes should your API return for these scenarios?

CONCEPT 200 (success with body), 201 (resource created), 204 (success, no body — e.g. a DELETE), 400 (malformed request), 401 (not authenticated), 403 (authenticated but not authorized), 404 (resource doesn't exist), 409 (conflict with current state, e.g. a duplicate unique key), 422 (well-formed but semantically invalid, e.g. a validation failure), 429 (rate limited), 500 (unhandled server error).

EXAMPLE A signup with an already-registered email should return 409, not 400 — the request was well-formed, it just conflicts with existing state.

ANALOGY These are the "reason codes" on a returned package — each one tells the sender exactly why, instead of one generic "failed."

SENIOR SIGNAL 401 vs. 403 and 400 vs. 422 are the two pairs candidates mix up most. Get those two distinctions right and you've covered the real test.

6 How do you design an API endpoint that might take 30 seconds to complete?

CONCEPT Don't hold the HTTP connection open for a 30-second operation. Use the async job pattern: POST kicks off the work and immediately returns 202 Accepted with a job ID, and the client polls GET /jobs/:id for status. Alternatively, use webhooks — the client registers a callback URL and the server POSTs the result when done, avoiding polling entirely.

EXAMPLE Video transcoding or large report generation — POST /reports returns 202 plus a job ID; the client polls GET /reports/:id until status is "done," then fetches the result.

ANALOGY Dropping off dry cleaning (get a claim ticket, come back later) instead of standing at the counter waiting for it to be pressed.

SENIOR SIGNAL Mention both patterns — polling AND webhooks — and when you'd choose each: webhooks for server-to-server, polling when the client can't expose a public endpoint.

7 What is HATEOAS and is it worth implementing?

CONCEPT Hypermedia As The Engine Of Application State — API responses include links to the next valid actions, so a GET /orders/5 response includes a "cancel" link only if the order is actually cancellable, and the client never hardcodes business rules about what's allowed next. Technically the purest form of REST, but in practice most teams skip it — it adds real implementation overhead for a benefit most clients never actually leverage.

EXAMPLE A payment gateway returning `_links: {refund: "/payments/5/refund"}` only when a payment is actually refundable, instead of the client guessing based on separate documentation.

ANALOGY A vending machine that only lights up the buttons for items in stock, instead of a static button panel with a separate paper list of what's sold out.

SENIOR SIGNAL Say "rarely implemented, high overhead" as your honest conclusion. Interviewers are checking whether you'll blindly praise a "correct" concept or give a grounded opinion.

8

How do you design backward-compatible API changes?

CONCEPT Add new fields, never remove existing ones. Make new fields optional with sensible defaults. Never change an existing field's type or meaning. Deprecate before removing — mark it in docs or headers, give a sunset date, monitor usage — rather than just deleting it.

EXAMPLE Adding `phone_number` as an optional new field to a user object is safe. Renaming `name` to `full_name` in place breaks every existing client parsing the old key.

ANALOGY Renovating a building while it's occupied — add a new wing, don't demolish a wall people are currently leaning against.

SENIOR SIGNAL "Add, don't change" as a one-line rule of thumb is exactly the takeaway interviewers want you to land on.

09 Database Design

VERY COMMON

Reveals whether you've designed a schema that survived production or just one that worked in dev. The N+1 problem, migrations, and sharding are the three most common deep dives.

1 What is database normalisation? What are 1NF, 2NF, 3NF?

CONCEPT 1NF: each column holds atomic values, no repeating groups crammed into one field. 2NF: 1NF, plus every non-key column depends on the **WHOLE** primary key — matters for composite keys, no partial dependency. 3NF: 2NF, plus non-key columns depend only on the primary key, not on each other — no transitive dependency.

EXAMPLE A table with `order_id`, `product_id`, `customer_name` where `customer_name` depends only on `order_id`, not on the composite key (`order_id`, `product_id`) — that's a 2NF violation. Move customer info to its own table.

ANALOGY 1NF is one item per box, not a bag of mixed items in one box. 3NF is every box's contents describing the box itself, not describing some other box.

SENIOR SIGNAL Use a concrete violation example instead of reciting definitions — it shows you can spot bad schema, not just define the rule.

2 When would you denormalise a database and what are the trade-offs?

CONCEPT Deliberately duplicating data, or pre-computing a join result, to make reads faster — at the cost of extra storage, slower and more complex writes (you must update every duplicate), and the risk of copies drifting out of sync. Only do it for a measured, real bottleneck, not preemptively.

EXAMPLE Storing a denormalized `comment_count` on a `posts` row instead of always running `COUNT(*)` on comments avoids a join or aggregate on every page load, at the cost of keeping that count in sync on every insert or delete.

ANALOGY Keeping a printed summary sheet next to a filing cabinet so you don't open every folder to count things — but now you must remember to update the sheet every time a folder changes.

SENIOR SIGNAL "Only for a measured bottleneck" is the discipline interviewers want to hear — denormalizing everything upfront is a red flag answer.

3 What is the N+1 query problem? How do you detect and fix it?

CONCEPT Fetching a list of `N` parent records, then issuing one additional query per record to fetch related data — `N+1` total queries instead of 1 or 2. Detect it by logging or counting queries per request, or via an APM, and spotting repeated queries differing only by an ID. Fix with a `JOIN`, or ORM eager loading — `JOIN FETCH` in JPA, `.include()` or `.with()` in other ORMs.

EXAMPLE Fetching 50 blog posts, then looping and calling `getAuthor(post.authorId)` inside the loop — 51 queries instead of one query with a `JOIN` on authors.

ANALOGY Sending someone to the grocery store 50 separate times, once per item on the list, instead of one trip with the whole list.

SENIOR SIGNAL Mention how you'd actually catch it in production — query logging, Hibernate statistics, an APM — not just "use eager loading."

4

When would you choose SQL over NoSQL and vice versa?

CONCEPT Choose based on access patterns and consistency needs, not raw scale. SQL fits complex relationships, multi-table joins, and strong ACID transactional needs with a relatively stable schema. NoSQL — document, key-value, graph, time-series — fits when the schema is flexible or evolving, access patterns are simple (fetch by key), or you need horizontal scale beyond what a single relational instance handles easily.

EXAMPLE An orders system with customers, products, inventory, and payments — heavy relationships, needs transactions — fits SQL. A product catalog with wildly varying attributes per category — shoes have sizes, TVs have resolutions — fits a document store.

ANALOGY SQL is a well-organized filing cabinet with cross-referenced folders. NoSQL is labeled shoeboxes you can grab by ID without needing to understand how they relate to each other.

SENIOR SIGNAL Say "it's about access patterns, not size" explicitly — that's the exact correction interviewers check for against the common "NoSQL equals big data" myth.

5

What is database sharding? What problems does it introduce?

CONCEPT Splitting a dataset across multiple independent DB instances by a shard key (e.g. `user_id % N`, or a range). Enables horizontal write scaling beyond one machine, but introduces real problems: cross-shard queries or joins become expensive or impossible without app-level fan-out, cross-shard transactions need something like 2PC or sagas, a poorly chosen key creates hot shards, and resharding later is a painful migration.

EXAMPLE Sharding an orders table by `customer_id` keeps "all of customer X's orders" on one shard — fast. But "top 10 products across all customers" now needs to query every shard and merge results.

ANALOGY Splitting one giant filing cabinet into ten smaller ones by last name — fast to find "Smith's file," painful to count how many files start with S across all ten cabinets.

SENIOR SIGNAL Mention the shard-key choice as the actual hard part — a bad key (low cardinality, skewed distribution) causes hot shards, which is usually the real interview follow-up.

6

How do you run a database migration with zero downtime?

CONCEPT The expand-contract pattern: 1) add the new column or table as nullable/additive, so a deploy doesn't break old code; 2) deploy code that dual-writes to old and new; 3) backfill historical rows into the new structure; 4) switch reads over to the new column; 5) once verified, remove the old column in a later deploy. Never run a blocking `ALTER TABLE` on a large table in prod without checking whether it locks.

EXAMPLE Renaming a column: add `email_address` (new), dual-write both columns, backfill old rows, cut reads over to the new column, then drop email weeks later once nothing references it.

ANALOGY Replacing a bridge while traffic still flows — build the new lane next to the old one, redirect traffic, then demolish the old lane, rather than closing the bridge to rebuild it.

SENIOR SIGNAL Naming all 5 expand-contract steps in order, unprompted, is a strong senior-level signal — most candidates stop at "add a column, migrate, remove old one" and miss the dual-write step.

7

What is connection pooling and what happens without it?

CONCEPT Reusing live DB connections instead of establishing a fresh TCP-plus-auth handshake per request. Without it, each request pays that overhead directly, and under load the DB's `max_connections` limit gets hit, causing

new connections — and their requests — to fail outright.

EXAMPLE A traffic spike without pooling can exhaust Postgres's default 100 `max_connections` in seconds, causing "too many connections" errors even though the DB itself isn't CPU-saturated.

ANALOGY A shared pool of rental cars at an airport versus everyone buying a new car the moment they land.

SENIOR SIGNAL Since this overlaps the concurrency topic, pivot into DB-specific tuning — pool size vs. the DB's `max_connections`, PgBouncer for connection multiplexing — to show more depth than a repeated answer.

8 What is a foreign key constraint and when would you skip it?

CONCEPT Enforces referential integrity at the DB level — you can't insert a row referencing a non-existent parent, and, depending on config, can't delete a parent with dependent children. Skip it when write throughput is critical (every insert pays a constraint-check cost), in sharded setups where parent and child might live on different shards (cross-shard FK isn't enforceable by the DB), or when the system deliberately accepts eventual consistency and handles integrity at the application or event layer instead.

EXAMPLE A high-volume event-logging table referencing a users table might skip the FK to avoid the per-insert check at scale, relying on application logic to keep `user_id` valid.

ANALOGY A bouncer checking ID at the door (FK) versus trusting people to just be who they say they are, because checking everyone would slow the line down too much.

SENIOR SIGNAL Give both a "keep it" and a "skip it" scenario — showing this isn't dogma either way is what a senior answer sounds like.

10 Production Debugging

ASKED EVERYWHERE

The most practical category. Interviewers ask scenario questions to see if you've actually been paged at 2am and solved a real problem. Structure every answer: observe, hypothesise, test, fix, prevent.

1 Your API response times doubled overnight. Walk me through debugging this.

CONCEPT Structure the investigation: check recent deployments and config changes first, since they're the most common cause, then the DB slow-query log, then infra metrics (CPU, memory, connection pool saturation), then external dependency latency, then cache hit rate. Narrow down to one of: a new code path doing extra work, a missing index, connection pool exhaustion, or a slow downstream dependency.

EXAMPLE A deploy 6 hours ago added a new field pulled via an unindexed query — the slow query log shows it immediately, EXPLAIN confirms a seq scan, and an index fixes it.

ANALOGY A doctor asks "what changed recently" before ordering tests — recent deploys are the equivalent of "did you eat something new."

SENIOR SIGNAL Lead with "what changed recently." It's the highest-signal first question and shows you don't jump straight to exotic causes.

2 Your database CPU is at 100%. What are the first 5 things you check?

CONCEPT In order: 1) currently running queries (pg_stat_activity / SHOW PROCESSLIST) to see what's executing right now; 2) the slow query log for recently expensive queries; 3) EXPLAIN on the top offenders, checking for a missing index or a bad join; 4) whether a recent schema or query change removed or invalidated an index; 5) lock contention and maintenance jobs running concurrently — Postgres autovacuum can spike CPU on large tables.

EXAMPLE pg_stat_activity shows the same query pattern, different IDs, running 200 times concurrently — a missing index on a newly added filter column, confirmed via EXPLAIN.

ANALOGY Checking a car's dashboard gauges (what's running right now) before popping the hood (log analysis) before rebuilding the engine (schema change).

SENIOR SIGNAL Naming the actual tools — pg_stat_activity, EXPLAIN — instead of "I'd investigate the queries" is what makes this answer credible.

3 How do you find a memory leak in a production Node.js service?

CONCEPT Take heap snapshots at intervals under load (Chrome DevTools via --inspect, or clinic.js) and diff them to see which object type keeps growing without being collected. Common root causes: event listeners registered but never removed, closures unintentionally holding references to large objects, or an in-memory cache/map with no eviction policy that just grows forever.

EXAMPLE A WebSocket server's memory climbs steadily over days — a heap snapshot diff shows a growing array of listener callbacks, because socket.on() was called on every reconnect without removing the old listener.

ANALOGY A guest list that only ever adds names and never removes anyone who left the party — eventually the list is bigger than the room.

SENIOR SIGNAL Naming the 3 common root causes unprompted — listeners, closures, unbounded caches — shows you've actually chased one before, not just read about --inspect.

4 What is distributed tracing and how does it help debug microservices?

CONCEPT A unique trace ID is generated at the entry point of a request and propagated through every downstream service call via headers. Each service adds its own "span" — start/end time and metadata — under that trace ID. Aggregated, you get a full waterfall view of the request's journey: which service was slow, what DB queries ran where, and exactly where an error originated, across service boundaries that logs alone can't stitch together.

EXAMPLE Jaeger, Zipkin, or OpenTelemetry showing a 3-second request was actually 2.8s spent in a single downstream inventory-service DB call, immediately narrowing the investigation instead of guessing across 5 services.

ANALOGY A package tracking number showing every hop it took, instead of each warehouse only knowing "a package passed through here" with no way to connect the dots.

SENIOR SIGNAL Emphasize that this solves correlating logs ACROSS services — that's the actual problem it exists to fix, not just "it shows request flow."

5 What is the difference between a metric, a log, and a trace?

CONCEPT A metric is an aggregated numeric measurement over time — p99 latency, error rate, CPU% — good for dashboards and alerting on trends, with no per-request detail. A log is a discrete, timestamped event with context — request ID, user, error message — good for investigating one specific incident. A trace is the end-to-end journey of one specific request across multiple services — good for pinpointing where in a distributed call chain something went wrong.

EXAMPLE A metric tells you error rate spiked to 5% at 2pm. Logs let you find one failing request and read its error message. A trace shows which specific downstream service that request got stuck in.

ANALOGY Metric is a fever thermometer reading — something's wrong, in aggregate. Log is the patient's symptom notes for one case. Trace is a full medical history following one patient through every department.

SENIOR SIGNAL Show they work together in sequence — metric alerts you, log narrows to a request, trace pinpoints the service — rather than describing them as three unrelated tools.

6 Your error rate is 1% but you can't reproduce the error. How do you debug it?

CONCEPT Filter error logs by the specific error type or message to find real failing requests, grab a request ID from one, and trace its full context — what user, what input, what code path, what's different from the 99% that succeed. If logging is insufficient to see the difference, add targeted logging to the suspected path and wait for it to reproduce, or replay real traffic against a debug build.

EXAMPLE 1% of failures all share one attribute — they're all from users with a null middle_name field that an unguarded string operation chokes on, found only after diffing failing request payloads against successful ones.

ANALOGY A doctor can't diagnose "sometimes people get sick" — they need the specific patients' charts to find what those cases have in common.

SENIOR SIGNAL Naming the move from "can't reproduce" to "diff the failing requests against the successful ones" is the actual technique — say that comparison step explicitly.

7 How do you debug a race condition that only happens in production?

CONCEPT Hard to reproduce locally because prod has real concurrency and timing that local dev doesn't. Add structured logging with precise timestamps and request or thread IDs at both read and write points of the suspected shared state, then analyze logs for overlapping time windows between two operations that shouldn't overlap. Once confirmed, fix with a DB-level atomic operation or a distributed lock, and validate by load-testing locally at higher concurrency to force the same interleaving.

EXAMPLE Two requests both updating the same inventory row within 5ms of each other in the logs, confirming a genuine race — fixed with `SELECT ... FOR UPDATE` or an atomic `UPDATE ... WHERE stock > 0`.

ANALOGY Watching security camera timestamps from two different doors to see if two people really entered the vault at the exact same second.

SENIOR SIGNAL Say you'd reproduce it locally via load testing at higher concurrency to confirm the fix. Many candidates stop at "I'd add a lock" without ever validating it actually worked.

8 What should a good post-mortem contain?

CONCEPT A timeline of what happened and when, factually. Impact — users and revenue affected, duration, severity. Root cause — the actual systemic gap, never "human error" as the final answer, since that stops the investigation one level too early. Contributing factors — what made it worse or let it go undetected longer. Action items — specific, owned, and dated. A blameless tone throughout, so people report honestly instead of covering up next time.

EXAMPLE "Engineer deployed a bad config" isn't the root cause. The root cause is "there was no automated validation or canary step that would have caught this before full rollout."

ANALOGY A plane crash investigation asks "why didn't the systems catch this," not just "who was flying" — the goal is fixing the system, not assigning blame.

SENIOR SIGNAL "Never 'human error' as the final answer" is the single most senior thing you can say here — it shows you think in systems, not individuals.
